



Keypoint & Pose

AICB-P2T4 · Ngày 04 · Chương 1: Hiểu về dữ liệu

Giảng viên

VinUniversity · Data Track

HÃY Suy Nghĩ...

“Ảnh chụp tài xế từ bên phải. Cánh tay đang cầm vô-lăng là tay trái hay tay phải của anh ta? Bạn có 3 giây – model sẽ học đúng cái bạn chọn.”

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Model Đọc Pose Thế Nào
2. Đi Sâu Vào Kiến Trúc Pose Estimators
3. Chuẩn Bị Dữ Liệu Pose
4. Kiểm Tra Chất Lượng

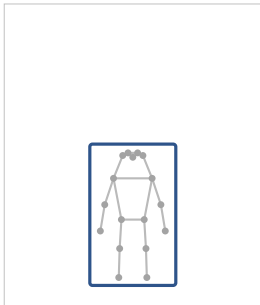
1. **Pose là gì** – 17 điểm có tên thay cho một cái box.
2. **Model đọc pose bằng cách nào** – heatmap, ngắn gọn.
3. **Chuẩn bị dữ liệu bằng CVAT Skeleton**, từng bước.
4. **Kiểm tra** – ba lượt, và một bản **visibility report**.

Ngày 3 bạn thêm *một* con số vào nhãn (`track_id`). Hôm nay bạn thêm **47** con số – và **17 quyết định** mà model không đoán hộ bạn được.

Model Đọc Pose Thế Nào

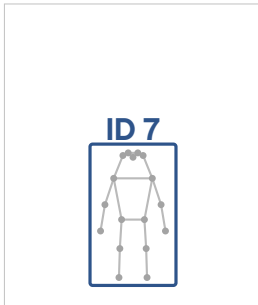
Từ một box sang 17 điểm có tên

01



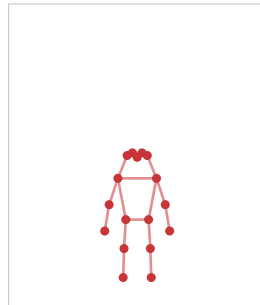
Ngày 2 – Box

4 con số



Ngày 3 – Box + ID

5 con số



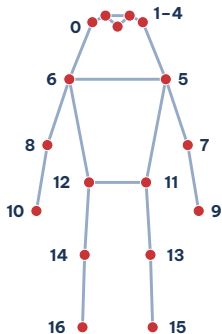
Ngày 4 – 17 điểm

51 con số

Cái box chỉ nói *người này ở đâu*. 17 điểm nói **người này đang làm gì** – đang giơ tay, đang cúi, đang cầm điện thoại.

	Ngày 2 · Detection	Ngày 3 · Tracking	Ngày 4 · Pose
Nhãn của một người	1 box	1 box + track_id	17 điểm có tên + cờ cho từng điểm
Số phải ghi	4	5	51 = $17 \times (x, y, v)$
Lỗi điển hình	box lệch	ID nhảy	trái/phải đảo nhau
Thước đo	IoU	– (Ngày 3 kiểm bằng <i>đếm ID</i>)	OKS (slide sau)

Mỗi ngày nhãn dày thêm một lớp – và lớp mới luôn là thứ model không tự suy ra được.



Thứ tự cố định, từ đầu xuống chân, trái trước phải sau:

- 0 nose – mũi
- 1--4 mắt trái, mắt phải, tai trái, tai phải
- 5--10 vai, khuỷu tay, cổ tay (trái rồi phải)
- 11--16 hông, đầu gối, cổ chân (trái rồi phải)

Đường nối là để cho *người nhìn*

Model chỉ học **17 cái chấm**. COCO nói rõ: 19 đường nối chỉ dùng để vẽ. Nhưng chính chúng là công cụ bắt lỗi nhanh nhất của bạn – xem lượt kiểm số 1.

Nguồn: COCO – person keypoints, 17 điểm / 19 cạnh

Visibility flag – ký hiệu v

Cổ tay tài xế nằm *sau vô-lăng*: bạn vẫn thấy cánh tay, vẫn biết cổ tay ở đâu, nhưng không nhìn thấy nó.

Vậy ghi gì vào file? COCO trả lời bằng **một con số thứ ba** cho mỗi điểm:

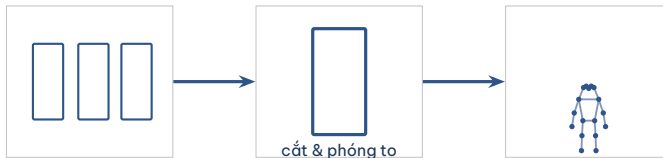
$v = 2$ – nhìn thấy rõ

$v = 1$ – bị che, *vẫn* đặt chấm

$v = 0$ – không gán nhãn (ra ngoài khung)

v không phải ghi chú. v là một phần ba của nhãn.

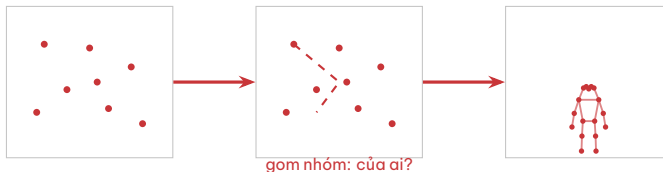
Top-down tìm người trước



3 người = 3 lượt chạy.

Chính xác hơn. Detector bỏ sót ai thì người đó không bao giờ có pose.

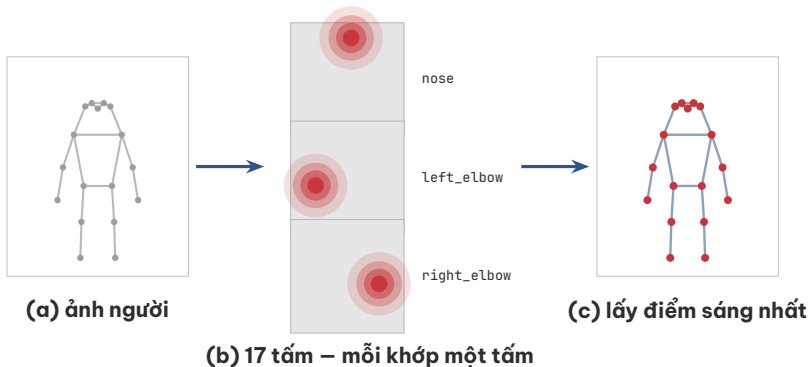
Bottom-up tìm khớp trước



1 lượt chạy, dù 1 hay 20 người.

Nhanh và ổn định. Nhưng đám đông chồng lấn thì gom nhầm.

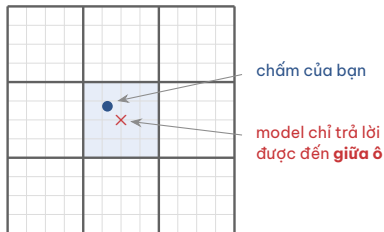
Top-down = phát cho mỗi người một tờ giấy rồi mới điền. **Bottom-up** = đổ hết cục áo ra bàn rồi mới phân xem cục nào của áo nào.



Mỗi tấm trả lời một câu: “**khớp này** chắc nằm ở đâu?”

Vệt càng nhoè = model càng **phân vân**. Một cặp tọa độ không nói được điều đó.

Khi huấn luyện, **chấm của bạn không được dùng nguyên**: nó bị *tô nhoè* thành đúng một vệt sáng như thế trên heatmap – vệt sáng đó mới là bài học của model.



- Với model dùng **heatmap**: heatmap đó **nhỏ hơn ảnh 4 lần** mỗi chiều – một ô của nó = 4×4 pixel ảnh.
- Nên model *không thể* trả lời chính xác hơn 4 pixel, dù nó có muốn.

Quy tắc để không phí 90 phút

Sai **2 pixel**: model gần như không nhận ra.

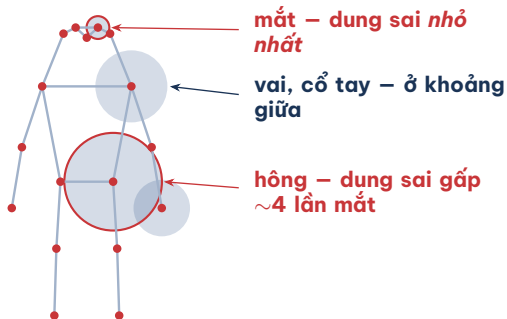
Sai **trái/phải**: model học sai vĩnh viễn.

Đúng khớp quan trọng hơn đúng pixel.

Model	Kiểu	Ý tưởng riêng	Dùng ở đâu
YOLO11-Pose	single-shot	một lượt ra cả box lẫn 17 điểm, dù bao nhiêu người. Không top-down, không bottom-up; đoán thẳng toạ độ, không qua heatmap	Lab hôm nay; chạy trên máy tính thường
OpenPose	bottom-up	PAF (<i>Part Affinity Fields</i>): vẽ mũi tên dọc cánh tay để biết cổ tay nối với khuỷu nào	Đám đông; vô địch COCO 2016
HRNet	top-down	<i>Không</i> thu nhỏ ảnh – giữ độ phân giải cao suốt quá trình	Khi cần chính xác nhất
MediaPipe	top-down	33 điểm (thêm bàn tay, bàn chân); chạy trên điện thoại	Trong cabin, thời gian thực

Nâng chất lượng *nhân* rẻ hơn nhiều so với nâng chất lượng *model* – yolo11x-pose tốn gấp ~ 27 lần phép tính, chỉ để thêm 19,5 mAP.

Bạn sẽ chạy yolo11n-pose trong lab – model nhỏ nhất trong bảng.



Cùng một sai số. Ở **mắt** là sai. Ở **hông** là đúng. Cách chấm điểm này gọi là **OKS** – *Object Keypoint Similarity*, IoU của pose.

Nguồn: vòng tròn vẽ theo per-keypoint sigma của COCO, phóng to cho dễ nhìn. Với một người cao ~100 px trong ảnh: dung sai tại mắt ~2,5 px, tại hông ~10,8 px

- COCO không *nghĩ ra* con số đó. Họ cho **nhiều người gán nhãn cùng 5.000 tấm ảnh**, rồi **đo** xem họ lệch nhau bao nhiêu ở từng khớp.
- Mắt, mũi, tai: ai cũng chấm gần giống nhau → **dung sai nhỏ**.
- Vai, gối, hông: không ai đồng ý với ai → **dung sai lớn**. Hông có dung sai gấp **4 lần** mắt.

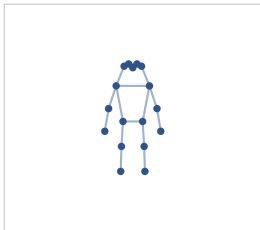
Vì sao hông khó nhất

Hông nằm *sau quần áo* – không có bề mặt nào để nhìn. Nó là một **ước lượng giải phẫu**, không phải một thứ nhìn thấy được.

Khớp không có bề mặt nhìn thấy được = khớp phải có luật viết sẵn trong guideline.

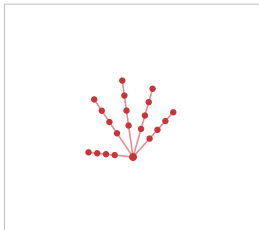
- Châu Âu đã **bắt buộc** xe mới có camera theo dõi tài xế. Hệ thống đó tên là **DMS** – *Driver Monitoring System*; bản theo dõi cả hành khách gọi là **OMS** – *Occupant Monitoring System*.
- “Có nhìn đường không?” → mắt, mũi, tai. “Tay có trên vô-lăng không?” → cổ tay + điểm bàn tay. “Dây an toàn đúng chưa?” → vai + hông.
- COCO-17 **không có bàn tay, không có chi tiết khuôn mặt**. Đó chính là lý do deliverable hôm nay có thêm **face/hand subset**.

Lưu ý: Trong cabin tài xế *ngồi*: ghế che hông và gối, vô-lăng che cổ tay **liên tục**. $v = 1$ là trạng thái bình thường ở đây, không phải ngoại lệ.



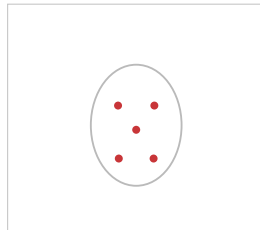
17 điểm thân

bộ COCO của hôm nay



21 điểm – một bàn tay

MediaPipe / COCO-WholeBody

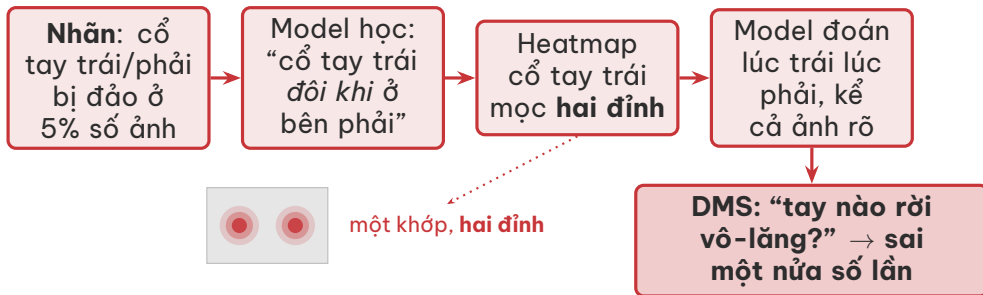


5 điểm mặt rút gọn

hai mắt · mũi · hai khoé miệng

Bàn tay *một* bên đã bằng 21 điểm – hơn cả bộ thân 17 điểm. Nên bộ này chỉ làm cho **5 ảnh**, không phải cả 20.

Đây là một skeleton label *riêng*, có topology riêng, dựng riêng trong CVAT – **không trộn vào bộ 17 điểm.**



Huấn luyện còn *lật ảnh* để tăng dữ liệu – nên nhãn đảo trái/phải được dạy cho model **hai lần**, ở cả hai chiều lật.

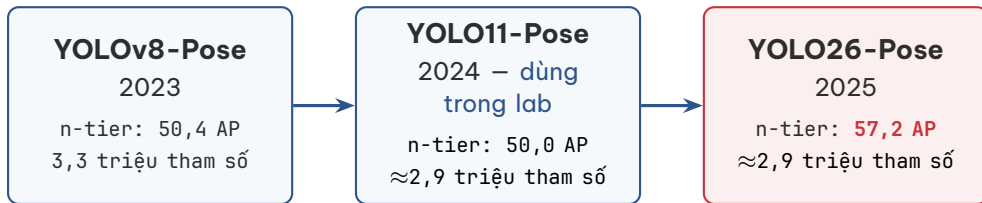
Đi Sâu Vào Kiến Trúc Pose Es- timators

Phần mở rộng – YOLO-Pose và ViTPose

02

- Bốn model vừa học (**YOLO11-Pose**, **OpenPose**, **HRNet**, **MediaPipe**) đủ để làm lab. Phần này đi sâu vào **hai** model đại diện cho hai hướng kiến trúc khác nhau.
- **YOLO-Pose**: một CNN **một lượt chạy**, ra thẳng box + 17 điểm – không top-down, không heatmap.
- **ViTPose**: bỏ hẳn CNN, dùng một **Vision Transformer tron** – kiến trúc đứng sau ChatGPT, áp vào ảnh.

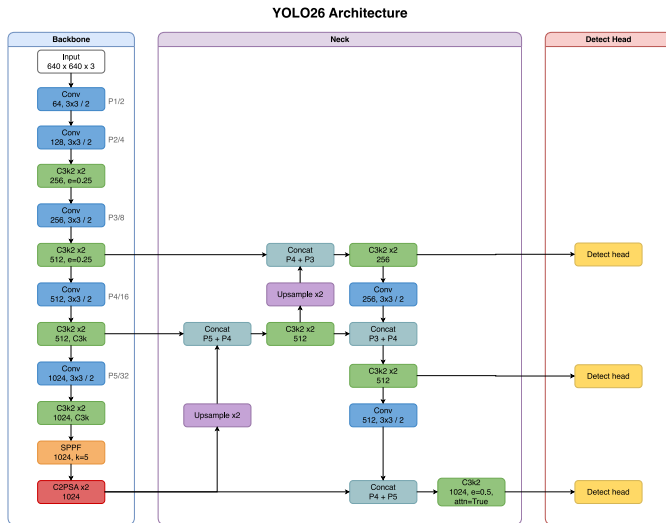
Đây là phần **đọc thêm**. Bạn vẫn nộp lab bằng yolo11n-pose, không đổi.



Cùng một ý tưởng suốt ba thế hệ: **một lượt chạy, ra cả box lẫn 17 điểm**, không cần chọn top-down hay bottom-up. Từ v8 sang 11: **nhẹ hơn, độ chính xác giữ nguyên**. Từ 11 sang 26: **+7,2 AP miễn phí** ở cùng kích thước model (Ultralytics tự công bố), nhờ bỏ bước NMS và một cách tính lại toạ độ chính xác hơn (RLE).

Lab hôm nay vẫn dùng yolo11n-pose – ổn định, có tài liệu tốt. Đây là bối cảnh, không phải lệnh đổi model.

Nguồn: docs.ultralytics.com – YOLO26, so sánh YOLO11 vs YOLOv8



Nguồn: Ultralytics, YOLO26 paper, arXiv:2606.03748, Fig. S1

- **Backbone** (trái): ảnh đi qua nhiều lớp Conv (giảm kích thước dần – $P1/2 \rightarrow P5/32$) xen kẽ khối C3k2. Cuối backbone: SPPF gom ngữ cảnh nhiều tầm nhìn khác nhau **rẻ tiền**, và C2PSA thêm một cơ chế **chú ý không gian** (attention) – giúp model để ý đúng vùng có người, kể cả người nhỏ hoặc bị che.
- **Neck** (giữa): trộn đặc trưng từ **nhiều tầng** backbone ($P3/P4/P5$) theo hai chiều – Upsample + Concat đi lên (để tầng nhỏ “mượn” ngữ cảnh từ tầng lớn), rồi Conv stride 2 đi xuống lại. Đây là lý do **một** model nhận ra được cả người ở gần lẫn người ở xa.
- **Head** (phải): mỗi tầng ($P3, P4, P5$) có một **Detect head** riêng. Bản pose thay head này bằng head dự đoán **box + 17 điểm**, vẫn dùng chung backbone/neck ở trên.

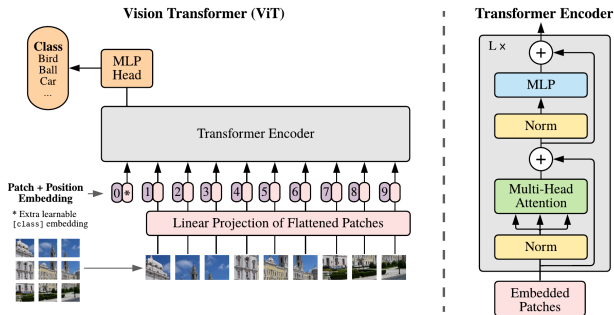
Hai điểm khác YOLO11 nằm ở chính cái head này

DFL-free: bỏ một bước tính toán khi giải mã tọa độ box – đơn giản hơn, dễ xuất sang phần cứng nhúng hơn. **NMS-free**: bỏ hẳn bước lọc trùng sau cùng – đúng thứ một hệ thống **thời gian thực** như DMS/OMS cần.

- Ảnh được chia thành một **lưới ô** ở ba tầng khác nhau (P3/P4/P5, từ neck đã học). Mỗi ô **tự** dự đoán: có người hay không, box lệch tâm ô bao nhiêu, và **17 tọa độ khớp** – tất cả trong *một* lần forward pass.
- **Anchor-free** nghĩa là gì: các bản YOLO cũ cần đặt sẵn vài “khung neo” (anchor box) mẫu ở mỗi ô rồi học độ lệch. YOLO hiện đại bỏ hẳn bước neo sẵn – mỗi ô tự đoán thẳng kích thước, đơn giản hoá huấn luyện.
- Vì sao nhanh: *một* lần chạy cho **cả ảnh**, không lặp lại cho từng người như top-down (HRNet đã học) – độ phức tạp **không tăng theo số người** trong ảnh.

Lưu ý: Đánh đổi: mỗi ô chỉ “nhìn” một vùng nhỏ, nên hai người đứng **rất sát nhau** có thể rơi vào cùng một ô. NMS (trước YOLO26) hoặc one-to-one matching (YOLO26) phải xử lý việc “ô này thuộc về ai”.

Vision Transformer là gì? – từ ảnh thành “từ vựng”



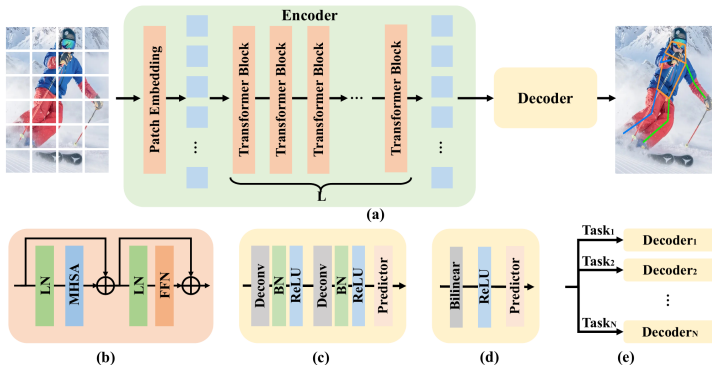
Ảnh bị cắt thành các **ô vuông nhỏ** (patch, vd 16×16 pixel) → mỗi patch **đuổi phẳng** thành một vector, giống một “từ” → cộng thêm **position embedding** (để model biết patch nào ở đâu) → cả chuỗi “từ” này đi vào một **Transformer Encoder** tiêu chuẩn – đúng kiến trúc đúng sau các mô hình ngôn ngữ như ChatGPT, chỉ đổi đầu vào từ chữ sang ảnh.

Nguồn: A. Dosovitskiy et al., *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*, ICLR 2021 – Figure 1

- Một CNN (HRNet, YOLO-Pose) chỉ nhìn từng **vùng nhỏ** một lúc (kernel 3×3 , 7×7 ...), phải **xếp chồng nhiều lớp** mới “thấy” được toàn ảnh.
- **Self-attention**: ngay từ lớp đầu tiên, mỗi patch được so sánh với *mọi* patch khác trong ảnh cùng lúc – patch ở bàn tay có thể “chú ý” thẳng đến patch ở khuôn mặt, dù cách xa nhau.
- Cái giá phải trả: so sánh mọi cặp patch với nhau tốn nhiều phép tính hơn một kernel nhỏ – đây là lý do ViT cần **nhiều dữ liệu và nhiều tham số** mới thắng được CNN.

Câu để nhớ

CNN học nhìn xa dần qua nhiều lớp. Transformer nhìn xa **ngay từ đầu** – đổi lại bằng nhiều phép tính hơn.



(a) Ảnh người → Patch Embedding → L Transformer Block **giống hệt nhau**, không có gì “may đo” riêng cho pose → Decoder. **(b)** Mỗi Transformer Block: chuẩn hoá (LN) → Multi-Head Self-Attention → cộng lại → chuẩn hoá → FFN → cộng lại. **(c)(d)** Decoder: chỉ 2 lớp deconvolution (hoặc 1 lớp upsample ở bản “simple”) + 1 lớp dự đoán – ra heatmap 17 khớp.

Nguồn: Y. Xu et al., ViTPose: Simple Vision Transformer Baselines for Human Pose Estimation, NeurIPS 2022 – Figure 2

ViTPose – một transformer “thường”

Mọi model vừa học (HRNet, YOLO-Pose) đều là một kiến trúc CNN được **thiết kế riêng** để giữ hoặc phục hồi độ phân giải không gian.

ViTPose (2022) đặt câu hỏi ngược lại: nếu dùng một **Vision Transformer** – loại transformer bình thường dùng để phân loại ảnh, không chỉnh sửa gì cho riêng bài toán pose – thì sao?

Câu trả lời: vẫn thắng, miễn là đủ dữ liệu và đủ tham số – từ 100 triệu đến **1 tỷ** tham số (L càng lớn, model càng mạnh), độ chính xác vẫn tăng đều.

Đơn giản hơn (không thiết kế riêng) lại mở rộng tốt hơn.

- ViTPose gốc: *một* backbone cho *một* bài toán (pose người trên COCO).
- **ViTPose++** (2023) dùng **MoE** – *Mixture of Experts*: vẫn *một* backbone dùng chung, nhưng thêm vài “chuyên gia” nhỏ riêng cho từng việc – một chuyên gia cho pose người, một cho pose **động vật**, một cho pose **toàn thân** (mặt + tay).
- Kết quả: *cùng một* model xử lý được nhiều loại pose khác nhau, mà **không chậm đi** – vì phần thân transformer vẫn dùng chung, chỉ các chuyên gia nhỏ là khác nhau.

Vì sao đây là hướng đi tự nhiên cho transformer

Một backbone, nhiều đầu ra chuyên biệt – đúng ý tưởng đã thấy ở YOLO-Pose (một backbone, ba tầng detect) – nhưng ViTPose++ áp nó cho **nhiều bài toán**, không chỉ nhiều tầng phân giải.

Nguồn: Y. Xu et al., ViTPose++: Vision Transformer for Generic Body Pose Estimation, TPAMI 2023

Chọn **một** pose estimator hiện đại (không nhất thiết phải là YOLO-Pose hay ViTPose – RTMPose, MediaPipe, MMPose, OpenPose...đều được).

1. **8 phút, tự nghiên cứu:** tìm trang chính chủ của model (GitHub, docs.ultralytics.com, hoặc paper) – đọc nó **dùng để làm gì**, tìm một notebook **Google Colab** có sẵn (nút “Open in Colab”).
2. **10–12 phút, tự chạy demo:** mở notebook trong Colab, chạy thử trên **một ảnh mẫu** (có sẵn hoặc ảnh của chính bạn).
3. **5 phút, ghi lại: 3 gạch đầu dòng** – đúng chỗ nào, sai/thiếu chỗ nào, chạy nhanh hay chậm.

Vì sao phải tự chạy, không chỉ đọc

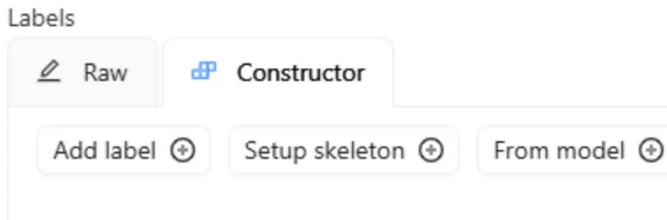
Đọc con số AP trên paper khác hẳn với **tự thấy** model đoán sai ở đâu trên một ảnh thật – kỹ năng bạn sẽ dùng suốt khoá.

Chuẩn Bị Dữ Liệu Pose

Skeleton trên CVAT: dựng một lần, dùng cho cả batch

- **CVAT Skeleton** – nơi *bạn* đặt 17 điểm.
-  **Ultralytics Pose** – model chấm thử một lần để bạn *đối chiếu*; không phải đáp án.
-  Script đếm cờ visibility – giảng viên phát sẵn, chỉ cần chạy.

Lưu ý: Giống Ngày 3: chạy model **trước** khi tự gán nhãn rất dễ khiến bạn tin nhầm vào model. Tự chấm trước, rồi mới đối chiếu.



1. Ở màn hình tạo task, bấm **Setup skeleton**.
2. **Add point** $\times 17$, kéo từng điểm về đúng khớp.
3. **Add edge** $\times 19$.
4. Chuột phải một điểm $\rightarrow$ **Configure** để **đặt tên** (left_wrist, right_wrist...).
5. **Submit & Open**.

Đường tắt: From model $\rightarrow$ Human pose estimation – CVAT dựng sẵn cả 17 điểm, đúng tên đúng thứ tự. **Gõ tay 17 cái tên là 17 cơ hội gõ sai.**

Nguồn: docs.cvat.ai – Annotation with skeletons (labels constructor)

topology – bộ **17 điểm** và **19 đường nối**, đúng tên và đúng thứ tự. Đổi một điểm là đổi topology.

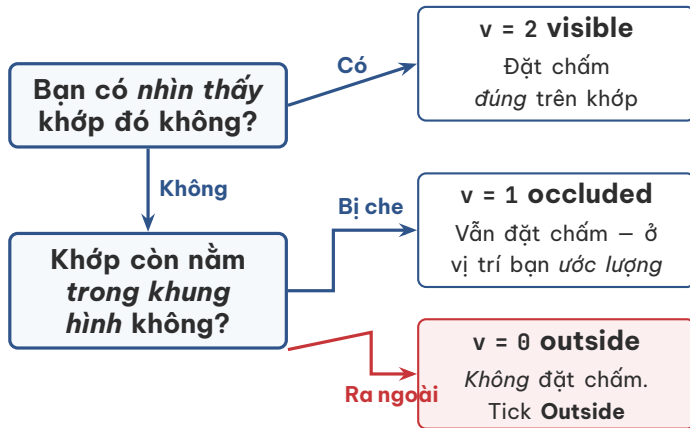
- CVAT nói thẳng: **không thể đổi cấu hình skeleton của một task đã tạo.**
- Đặt thiếu một điểm, đặt sai thứ tự → cả task làm lại từ đầu.
- Nên: dựng *một* lần, rồi **Download skeleton** ra file .SVG – hoặc chép JSON ở tab **Raw** – và **Upload** lại cho mọi task sau.

Đây chính là “landmark consistency”

Nhãn của bạn là một **bản hợp đồng**, không phải sự thật về cơ thể người. 17 điểm hay 33 điểm đều đúng; *trộn hai bộ* mới là sai.

1. Mở job → thanh công cụ → **Draw new skeleton**.
2. Chọn **Shape** (ảnh tĩnh) hoặc **Track** (clip trong cabin). **Chọn sai là làm lại.**
3. Vẽ – CVAT thả cả bộ 17 điểm xuống cùng lúc, trong một cái box.
4. Kéo/xoay cả box cho khớp thô, rồi mới **kéo từng điểm** về đúng khớp.
5. Đặt cờ cho mọi điểm không nhìn thấy rõ (slide sau), rồi bấm **Save**.

Lưu ý: Ở chế độ **Track**, keyframe và interpolation của Ngày 3 áp dụng cho *từng điểm một*. Nội suy 17 điểm là **17 cơ hội trôi** giữa hai keyframe – đặt keyframe dày hơn hẳn so với khi vẽ box.



Bị che thì vẫn đặt chấm. Xóa nó đi là dạy model rằng khuỷu tay *biến mất* mỗi khi khoanh tay trước ngực.

Với **5 tình huống** dưới đây, tự viết ra: bạn chọn $v = 0, 1$ hay 2 ? Kèm **một câu** lý do.

1. Cổ tay phải bị chính đùi của người đó che khuất, khi họ đặt tay xuống ghế.
2. Đầu gối phải bị cắt đúng ở mép dưới khung hình.
3. Vai phải bị ba-lô của người ngồi ghế sau che kín hoàn toàn.
4. Tai trái bị tóc che một phần, nhưng bạn vẫn đoán khá chắc vị trí.
5. Người ngồi xa phía sau – ảnh chỉ chụp được từ đầu đến ngang hông.

Lưu ý: 8 phút tự làm, **không** lật lại slide trước. 5 phút sau: so với 2–3 bạn ngồi gần – khớp nào cả nhóm lệch nhau nhiều nhất?

1. $v = 1$ – còn trong khung, chỉ bị *vật thể khác* (chính cơ thể người đó) che. Vẫn ước lượng và đặt chấm.
2. $v = 0$ – ra ngoài mép ảnh. Lý do bị che hay bị cắt không quan trọng, chỉ cần còn *trong khung* hay không.
3. $v = 1$ – bị vật thể che, không phải ra ngoài khung. Ba-lô không phải mép ảnh.
4. $v = 2$ (gợi ý) – “đoán khá chắc” vẫn là *nhìn thấy*. Nhưng đây là ca **gây tranh cãi nhất** trong 5 ca.
5. $v = 0$ cho đầu gối/cổ chân – ngoài khung hình, dù lý do là khoảng cách xa chứ không phải vật che.

Nếu cả nhóm chia phe ở câu 4

Không sao – đúng như slide trước đã nói: tai bị tóc/mũ che là một trong những ca cần **ảnh mẫu trong guideline**, không phải một câu văn chung chung. Chia phe ở câu này là bằng chứng guideline lớp bạn cần nói rõ hơn.

CVAT – cột bên phải

file labels/*.txt nộp cho model

Nhìn thấy rõ ☐ Occluded ☐ Outside

... 0.42 0.31 **2** ... vẫn chấm điểm

Bị che sau
vô-lăng

☒ Occluded ☐ Outside



... 0.55 0.29 **1** ... vẫn chấm điểm

Ra ngoài
khung

☐ Occluded ☒ Outside

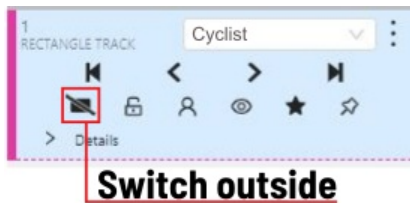
... 0.00 0.00 **0** ... **bị loại khỏi OKS**

Hai cái tick đó **không** phải ghi chú cho người sau đọc. Chúng *là* con số thứ ba của mỗi khớp – và $v = 0$ **khiến khớp đó biến mất khỏi bảng điểm**, không ai được và cũng không ai mất.

Trong CVAT	Nghĩa là gì	Ra file	Phím tắt
(không tick gì)	Nhìn thấy rõ	$v = 2$	—
Occluded	Còn trong khung, bị vật khác che	$v = 1$	q hoặc /
Outside	Ra ngoài mép khung hình	$v = 0$	o
Hidden	chỉ để bạn đỡ rối mắt	không ghi gì	h

Lưu ý: **Hidden** trông y hệt **Outside** trên màn hình và **không được lưu**. Bấm nhầm là điểm đó vẫn xuất ra $v = 2$ ở vị trí cũ – sai mà không có một dòng cảnh báo nào.

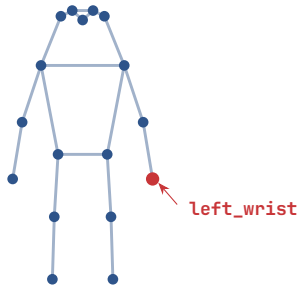
Thứ tự trong code CVAT: outside thắng occluded. Tick cả hai thì xuất ra $v = 0$.



- Di chuột lên **một điểm** rồi bấm phím → chỉ điểm đó đổi cờ.
- Di chuột lên **cái box bao** rồi bấm phím → cả skeleton đổi cờ.
- Quen từ Ngày 3: o = outside, k = keyframe.
- Thêm đúng **một** phím mới: **q = occluded**.
- Cột bên phải có mục xổ xuống liệt kê **từng điểm** – sửa cờ ở đó nhanh hơn mò trên ảnh.

Ngày 3 bạn bấm chúng cho *cả một cái box*. Hôm nay chúng áp cho *từng khớp*.

Nguồn: docs.cvat.ai – Editing skeletons on the canvas



người quay mặt về phía camera

Người này **quay mặt** về phía bạn.

Tay *trái* của anh ta xuất hiện ở **bên phải của bức ảnh**.

Vẫn ghi là left_wrist.

Cách kiểm tra trong 1 giây

Tự tưởng tượng bạn *đứng vào vị trí người đó* rồi giơ tay trái lên.

COCO đã đo: ngay cả trong bộ dữ liệu chuyên nghiệp, **1-2% nhãn vẫn sai** – phần lớn do nhầm trái/phải *hoặc* gán nhầm sang người bên cạnh.

1. Khớp **sau thân mình / sau vô-lăng / sau người khác** → occluded, vẫn đặt chấm ước lượng.
2. Khớp **bị cắt ở mép ảnh** → outside. Kéo chấm ra ngoài ảnh *không* thay được cờ này.
3. **Hông** – không nhìn thấy trên bất kỳ người mặc quần áo nào. Guideline phải có *ảnh mẫu*, không phải một câu văn.

4. **Tai** khi tóc hoặc mũ bảo hiểm che, **mũi** khi chụp nghiêng, **vai** khi quay túi che.
5. **Người nhỏ đến mức nào thì không gán nữa?** Đặt một ngưỡng (ví dụ: thấy rõ ≥ 3 khớp).
6. **Hai người chồng lên nhau:** làm xong hẳn người này rồi mới sang người kia.

Điểm chung của cả sáu: **luôn đủ 17 điểm cho mọi người**. Điểm không dùng được thì **gán cờ, không bao giờ xóa**.

- Chọn **COCO Keypoints 1.0** – định dạng của deliverable hôm nay. (**Ultralytics YOLO Pose 1.0** là bản để huấn luyện.)
- **Đừng chọn YOLO 1.1 thường** – nó *chỉ* xuất box. 17 điểm biến mất: không lỗi, không cảnh báo. Đúng cái bẫy của Ngày 3.
- Nếu xuất YOLO Pose: trong `data.yaml` phải thấy `kpt_shape: [17, 3]`. Là `[17, 2]` thì **toàn bộ cờ visibility đã bị vứt**.

Mở file ra và đếm – mỗi định dạng một phép đếm

COCO Keypoints (bản nộp): mảng keypoints của mỗi người phải có $17 \times 3 = \mathbf{51}$ số.

Ultralytics YOLO Pose: một dòng nhãn phải có $5 + 51 = \mathbf{56}$ số.

Đếm ra 34 (hoặc 39) → làm lại *export*, đừng làm lại nhãn.

- ☒ Skeleton label lấy từ file .svg chung – không tự dựng lại
- ☒ Mọi người đều có **đủ 17 điểm**; điểm không dùng được thì gắn cờ
- ☒ Trái/phải tính theo **cơ thể người**, không theo bức ảnh
- ☒ Khớp bị che $\rightarrow$ q (occluded) và **vẫn đặt chấm**
- ☒ Khớp ra khỏi khung $\rightarrow$ o (outside), **không** đặt chấm
- ☐ **Không dùng h (Hidden)** – nó không được lưu vào file
- ☐ Chưa chỉnh đến từng pixel – **đúng khớp quan trọng hơn 2 pixel**

Kiểm Tra Chất Lượng

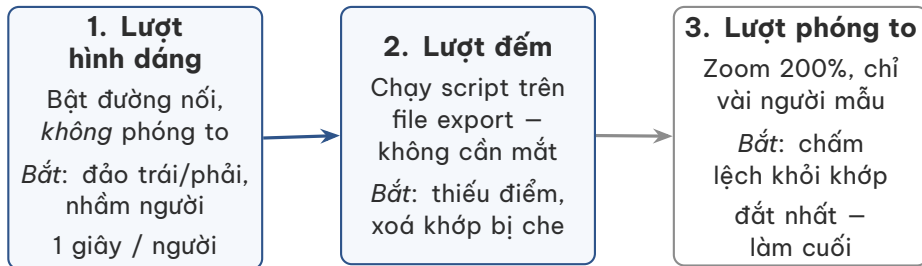
Ba lượt · bảy lỗi · một bản visibility report

04

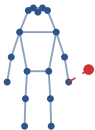
“

“Dữ liệu thật vẫn còn 1–2% nhãn ngoại lệ – phần lớn do người gán nhãn nhầm trái với phải, hoặc gán nhầm sang người đứng bên cạnh.”

– **COCO – Keypoint Evaluation**, nói về chính bộ dữ liệu của họ



Một bộ xương người *gần như* không bao giờ tự cắt chéo ở thân – trừ khi người đó **vặn mình** (quay sang ghế sau). Thấy cắt chéo thì **ngghi ngờ trước**, kiểm lại pose rồi mới sửa.



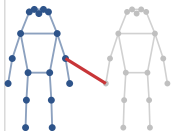
1. Lệch nhẹ

chấm gần đúng khớp. Sửa được, ít hại.



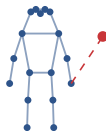
2. Đảo trái/phải

xương **cắt chéo** ở thân. Nguy hiểm nhất.



3. Nhầm người

xương **kéo dài** sang cơ thể bên cạnh.

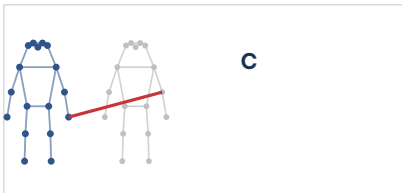
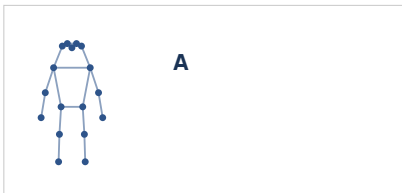


4. Trượt hân

chấm vào chỗ không có khớp nào.

Điều bất ngờ: **lỗi đảo trái/phải không xảy ra ở ảnh khó**. Nó xảy ra ở ảnh dễ, rõ ràng – lúc bạn đang làm nhanh.

Nguồn: phân loại lỗi theo Ronchi & Perona, *Benchmarking and Error Diagnosis in Multi-Instance Pose Estimation*, ICCV 2017



Với mỗi hình **A–D**: viết ra – **đúng** hay **có lỗi**? Nếu có lỗi, thuộc loại nào trong 4 loại vừa học? Không cần sửa, chỉ cần gọi đúng tên. **8 phút** tự làm, **5–7 phút** công bố đáp án.

- **A – Đúng, không lỗi.** Bộ khung không có đường nào cắt chéo hay lệch bất thường.
- **B – Đảo trái/phải.** Xương **cắt chéo ở chân** (đầu gối nổi chéo sang mắt cá chân đối diện) – cùng bản chất với ca cắt chéo ở vai/hông đã học, chỉ khác vị trí trên cơ thể.
- **C – Nhầm người.** Một khớp **kéo dài sang cơ thể bên cạnh** – dấu hiệu luôn là một đường nối bất thường dài, đi ra khỏi vùng thân người.
- **D – Trượt hần.** Một điểm nằm ở chỗ **không có khớp nào** – không cắt chéo, không kéo dài, chỉ đơn giản sai vị trí.

Nếu bạn đoán sai ở A

Đoán có lỗi khi không có lỗi cũng là một sai lầm thật – trong QC, việc này gọi là **false positive**: sửa một chỗ vốn đã đúng, tốn thời gian mà không tăng chất lượng.

	Lỗi	Trông như thế nào	Tìm bằng	Sửa
1	Đảo trái/phải	Xương cắt chéo ở vai hoặc hông	Lướt hình dáng	Đổi lại hai điểm
2	Nhầm người	Xương kéo sang cơ thể khác	Lướt hình dáng	Đặt lại điểm
3	Xoá khớp bị che	num_keypoints – số khớp có $v > 0$ – thấp hơn 17 ở hầu hết người	Lướt đếm	Gán lại với $v = 1$

*Bốn lỗi còn lại – thiếu/thừa điểm, cờ không nhất quán, dùng **Hidden** làm dữ liệu, chấm lệch khớp – nằm trong bản guideline in sẵn phát cho lớp.*

Một bảng đếm duy nhất, mỗi khớp một dòng: bao nhiêu $v=2$, bao nhiêu $v=1$, bao nhiêu $v=0$.

- Hai người có tỉ lệ $v=1$ *lệch hẳn nhau*: họ bất đồng về **guideline**, không phải về bức ảnh. Một khớp có $v=0$ nhiều bất thường cũng vậy – mọi người đang dùng **Outside** ở chỗ đáng lẽ là **Occluded**.
- File COCO ghi sẵn `num_keypoints` = số khớp có $v > 0$ của *mỗi người* – đủ để bắt lỗi “xóa khớp bị che”. **Bảng đếm theo từng khớp thì phải chạy script** giảng viên phát.

So report trước, so pixel sau

Bài học kiểm chéo của Ngày 3, viết lại cho pose: **nghi guideline trước, đừng nghi người**. Bảng đếm trả lời trong 2 phút; soi từng chấm mất 2 giờ.

Kiểm chéo với bạn cùng nhóm

Hai người gán **cùng một ảnh, độc lập**.

So **visibility report** trước. Chỗ nào lệch nhiều là chỗ guideline chưa nói rõ.

Rồi mới bật đường nổi lên và so hình dáng từng người.

CVAT tự chấm bằng OKS

Actions → **Quality control** – CVAT dùng đúng **OKS** của COCO để so với bản Ground Truth.

Chỉnh **Point size** = bán kính dung sai quanh mỗi điểm.

Cảnh báo: skeleton nào bị gán **Outside toàn bộ** sẽ bị *loại khỏi báo cáo*. Cờ đặt sai thì điểm số cũng sai theo.

- ✓ Mọi người trong ảnh đều có **đủ 17 điểm**, không ai bị thiếu
- ✓ Bật đường nối: **không có xương nào cắt chéo** ở vai hoặc hông
- ✓ Không có xương nào **kéo dài** sang một cơ thể khác
- ✓ Khớp bị che dùng $v=1$ (có chấm), không phải $v=0$
- ✓ Export đúng định dạng: **COCO Keypoints** – mảng keypoints có **51** số mỗi người (bản YOLO Pose: **56** số một dòng)
- ✓ **Visibility report** đã nộp, và hai bản đã được đặt cạnh nhau
- ✓ Mọi ca không rõ đều được **ghi lại** – không ai tự quyết rồi quên

LAB #4

Mục tiêu: Dựng **một** skeleton label 17 điểm trong CVAT rồi tái sử dụng bằng file .svg. Gán nhãn **20 ảnh trong cabin**: đủ 17 điểm cho mỗi người, cờ visibility cho từng điểm. Rồi dựng *một skeleton label thứ hai* – **21 điểm bàn tay + 5 điểm mặt** – và làm cho **5 ảnh**. Cuối cùng chạy yolo11n-pose trên cùng bộ ảnh và đối chiếu với bản bạn tự làm.

Deliverable: **17-point body pose set + face/hand subset** (export **COCO Keypoints 1.0**) + **visibility report** (bảng đếm theo từng khớp) + reviewer checklist đã điền

Thời gian: 90 phút

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 Nhấn pose = **17 điểm có tên** theo một topology *cố định*. Dựng label *một lần*, tái dùng bằng .SVG – topology bị khoá ngay khi task được tạo.
- 2 Mỗi điểm có **ba trạng thái**. Nhìn thấy $\rightarrow v=2$. Bị che $\rightarrow v=1$, *vẫn đặt chấm*. Ngoài khung $\rightarrow v=0$, không đặt chấm. **Hidden không phải dữ liệu**.
- 3 Mỗi khớp có **bán kính dung sai riêng**, đo từ chính sự bất đồng giữa người với người. Nên: **đúng khớp quan trọng hơn đúng pixel**.
- 4 Lỗi nguy hiểm nhất là **đảo trái/phải** – và nó xảy ra ở ảnh *đễ*. Bắt nó bằng **lượt hình dáng**: xương người *gần như* không bao giờ tự cắt chéo ở thân.

Bài tiếp theo

Ngày 05 – Segmentation

“Polygon và mask: từ 17 cái chấm sang việc tô kín từng pixel của một vật thể.”

Bài tập về nhà

- Nộp pose set + face/hand subset + visibility report trước buổi sau
- Pose hôm nay là nền cho **DMS/OMS** ở Ngày 10
- Nghĩ trước: ranh giới của vật thể *bị che một nửa* nên vẽ đến đâu?

1. COCO – *Data format* (cờ visibility $v = 0/1/2$). cocodataset.org/#format-data
2. COCO – *Keypoint evaluation* (OKS, per-keypoint sigma, 1–2% ngoại lệ).
cocodataset.org/#keypoints-eval
3. CVAT documentation – *Annotation with skeletons và Quality control*. docs.cvat.ai
4. Ultralytics documentation – *Pose Estimation và Pose Datasets*.
docs.ultralytics.com/tasks/pose
5. Z. Cao et al., *Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields*, CVPR 2017.
6. K. Sun et al., *Deep High-Resolution Representation Learning for Human Pose Estimation*, CVPR 2019.
7. M. R. Ronchi, P. Perona, *Benchmarking and Error Diagnosis in Multi-Instance Pose Estimation*, ICCV 2017.
8. V. Bazarevsky et al., *BlazePose: On-device Real-time Body Pose Tracking*, 2020.
9. J. D. Ortega et al., *DMD: A Large-Scale Multi-Modal Driver Monitoring Dataset*, 2020.
dmd.vicomtech.org
10. Ultralytics documentation – *YOLO26, YOLO11 vs YOLOv8 Comparison*.
docs.ultralytics.com/models/yolo26
11. A. Dosovitskiy et al., *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale (ViT)*, ICLR 2021, arxiv.org/abs/2010.11929

Hỏi & Đáp

*Câu hỏi mở: trong cabin, khuôn mặt là tín hiệu
– không thể làm mờ như dữ liệu ngoài phố.
Vậy bảo vệ người trong ảnh bằng cách nào?*



Cảm ơn!

Ngày 04 – Keypoint & Pose: 17 điểm và ba trạng thái

Slide và lab: L2L3-DataAnnotation/Day4

Ngày mai: không còn chấm nữa – tô kín từng pixel.